Week#05

PROGRAM FLOW CONTROL INSTRUCTIONS

Implementation of the Program Flow Control using "JMP" and "LOOP" instructions in Assembly language.

I. Program Flow Control using JMP Instruction

Program flow is controlled by decision making. Decisions are based on either certain conditions or no conditions. It is often necessary to transfer the program control to different location. In 8086/88 Assembly language, this transfer of control is executed by JMP instructions. These jumps can be intrasegment (or within the segment) and intersegment (or outside the segment). Intrasegment jumps are termed as NEAR jumps and the intersegment jumps are termed as FAR jumps. There are two basic types of jumps; conditional and unconditional. Conditional jumps are used to transfer the control based on occurrence of certain condition, whereas unconditional jumps do not need any condition to meet. In general, the jump instructions can be forward jump or backward jump from a location where the jump is initiated.

a) Unconditional Jump

The basic instruction that transfers control to another point in the program is JMP. The basic syntax of JMP instruction:

JMP label

A label in a program can be declared with a name ended with "**:**". Label can be any character combination but it must not be a keyword and must not start with a number, for example here are 3 legal label definitions:

- **label1:**
- **LABEL2:**
- **abc:**

Here's an example of JMP instruction:

```
        mov   ax, 5          ; set ax to 5.
        mov   bx, 2          ; set bx to 2.
        jmp   calc           ; go to 'calc'.
back:   jmp stop             ; go to 'stop'.
calc:
        add   ax, bx         ; add bx to ax.
        jmp   back           ; go 'back'.
stop:
        mov ah, 4ch          ; return to operating system.
        int 21h
```

Unconditional Jumps are categorized as SHORT, NEAR and FAR jumps.

SHORT JUMP allows a label of one byte long. The address of target location mentioned with the label is between -128 to +127 bytes of memory relative to the current location of IP register. It can transfer control 128 bytes (0 to +127) forward and 128 bytes (-1 to -128) backward. It syntax is **JMP SHORT<Label>**.

NEAR JUMP (the default jump) allows a label of two bytes long. It can transfer control 32768 bytes (0 to +32767) forward and 32768 bytes (-1 to -32768) backward. It is an intra-segment jump i.e. the jump is within the current code segment. It syntax is **JMP <Label>**. The target address can be any of the following addressing modes:

- Direct: JMP label,
- Register indirect: JMP BX,
- Memory indirect: JMP [DI]

FAR JUMP has the syntax **JMP FAT PTR <Label>**. It allows to transfer control out of the current code segment i.e., both the CS and IP is replaced with new values.

b) Conditional Jump:

The conditional jumps transfer control only when some condition meets, unlike JMP instruction that does an unconditional jump. These instructions are divided in three groups,

- first group just test single flag,
- second compares signed numbers, and
- third compares unsigned numbers.

Table I, II, and III show the different conditional jumps available.

Table I: Conditional Jump instructions to test single flag

Instruction	Description	Condition	Alternates
JZ , JE	Jump if Zero (Equal).	ZF = 1	JNZ, JNE
JC , JB, JNAE	Jump if Carry (Below, Not Above Equal).	CF = 1	JNC, JNB, JAE
JS	Jump if Sign.	SF = 1	JNS
JO	Jump if Overflow.	OF = 1	JNO
JPE, JP	Jump if Parity Even.	PF = 1	JPO
JNZ , JNE	Jump if Not Zero (Not Equal).	ZF = 0	JZ, JE
JNC , JNB, JAE	Jump if Not Carry (Not Below, Above Equal).	CF = 0	JC, JB, JNAE
JNS	Jump if Not Sign.	SF = 0	JS
JNO	Jump if Not Overflow.	OF = 0	JO
JPO, JNP	Jump if Parity Odd (No Parity).	PF = 0	JPE, JP

Table II: Conditional Jump instructions for Signed numbers

Instruction	Description	Condition	Alternates
JE , JZ	Jump if Equal (=). Jump if Zero.	ZF = 1	JNE, JNZ
JNE , JNZ	Jump if Not Equal (< >). Jump if Not Zero.	ZF = 0	JE, JZ
JG , JNLE	Jump if Greater (>). Jump if Not Less or Equal (not <=).	ZF = 0 and SF = OF	JNG, JLE
JL , JNGE	Jump if Less (<). Jump if Not Greater or Equal (not >=).	SF < > OF	JNL, JGE
JGE , JNL	Jump if Greater or Equal (>=). Jump if Not Less (not <).	SF = OF	JNGE, JL
JLE , JNG	Jump if Less or Equal (<=). Jump if Not Greater (not >).	ZF = 1 or SF < > OF	JNLE, JG

Table III: Conditional Jump instructions for Unsigned numbers

Instruction	Description	Condition	Alternates
JE , JZ	Jump if Equal (=). Jump if Zero.	ZF = 1	JNE, JNZ
JNE , JNZ	Jump if Not Equal (<>). Jump if Not Zero.	ZF = 0	JE, JZ
JA , JNBE	Jump if Above (>). Jump if Not Below or Equal (not <=).	CF = 0 and ZF = 0	JNA, JBE
JB , JNAE, JC	Jump if Below (<). Jump if Not Above or Equal (not >=). Jump if Carry.	CF = 1	JNB, JAE, JNC
JAE , JNB, JNC	Jump if Above or Equal (>=). Jump if Not Below (not <). Jump if Not Carry.	CF = 0	JNAE, JB
JBE , JNA	Jump if Below or Equal (<=). Jump if Not Above (not >).	CF = 1 or ZF = 1	JNBE, JA

Generally, when it is required to compare numeric values CMP instruction is used (it does the same as SUB (subtract) instruction, but does not keep the result, just affects the flags). The logic is very simple, for example:
It is required to compare 5 and 2,

5 - 2 = 3

the result is not zero (Zero Flag is set to 0).
Another example:
It is required to compare 7 and 7,

7 - 7 = 0

the result is zero! (Zero Flag is set to 1 and JZ or JE will do the jump).

II. Program Flow Control using LOOP Instruction

Loop instruction is special type of program flow control for repetition of an instruction or block of instructions. The Loop is implemented by using the command "LOOP" followed by the label of the instruction where to jump to. The maximum number of iterations (repetitions) is defined in the register CX. Table IV shows different loop instructions available.

The LOOP instruction first decrements the value of count defined in register CX and checks the value of CX register. If the CX register contains a non-zero value the loop continues otherwise the loop stops. For example, following code will add 1 in the value contained in register AL for 10 times.

```
        MOV CX, 10      ;CX=max. count value i.e.; 10
        MOV AL, 0       ;Initializing AL with zero
SUM:    ADD AL, 1       ;Add 1 to AL
        LOOP SUM        ;DEC CX and JNZ SUM
```

The alternate of the LOOP instruction can be a combination of two commands DEC and JNZ. To demonstrate this the above code can be written as:

Table IV: LOOP instructions

Instruction	Operation and Jump Condition	Alternates
LOOP	decrease cx, jump to label if cx not zero.	DEC CX and JCXZ
LOOPE	decrease cx, jump to label if cx not zero and equal (zf = 1).	LOOPNE
LOOPNE	decrease cx, jump to label if cx not zero and not equal (zf = 0).	LOOPE
LOOPNZ	decrease cx, jump to label if cx not zero and zf = 0.	LOOPZ
LOOPZ	decrease cx, jump to label if cx not zero and zf = 1.	LOOPNZ
JCXZ	jump to label if cx is zero.	OR CX, CX and JNZ

```
        MOV CX, 10      ;CX=max. count value i.e.; 10
        MOV AL, 0       ;Initializing AL with zero
SUM:    ADD AL, 1       ;Add 1 to AL
        DEC CX          ;CX=CX-1
        JNZ SUM         ;If zero flag is not set i.e.; CX != 0, jump to SUM
```

It is also to be noted that the count initialization for the CX value is done outside the loop.
Loops can also be used to insert delays, where necessary, in a Program. Each instruction has assigned a time to execute in the instruction set. The time to execute depends upon the machine cycles it takes to complete that particular instruction by the microprocessor. For example, to generate a delay of 1ms, LOOP instruction can be used to loop a NOP (no operation) instruction for number of times.